Sistema de gestión de rutinas de entrenamiento personalizadas

Integrantes:

- Miguel Margas – 21.020.364-8
- Andrés Olguín – 21.500.844-4

Ciudad: Viña del Mar

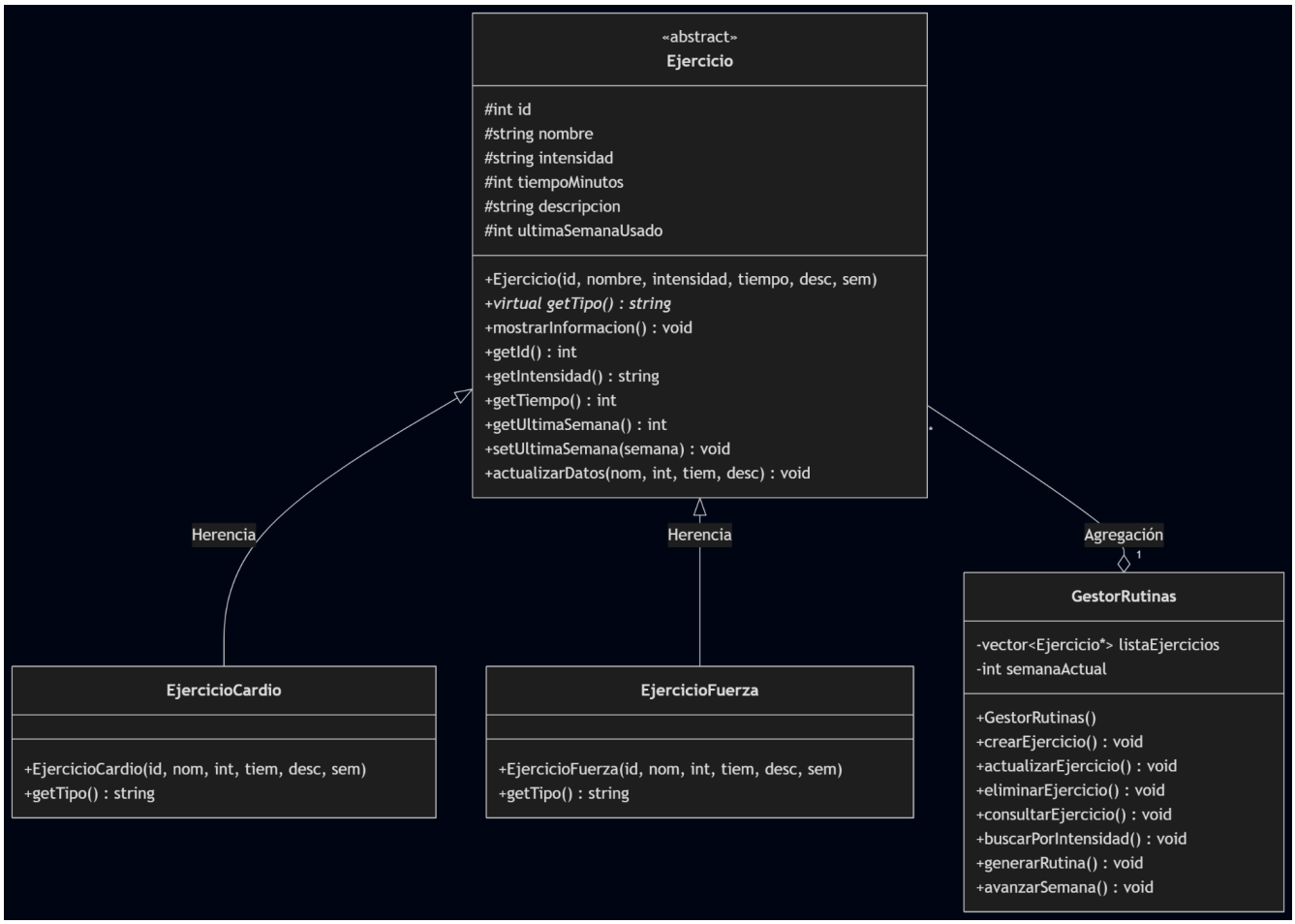
Fecha de entrega: 16 de abril de 2026

Introducción

El informe describe el diseño e implementación de sistema de software que se desarrolló en C++ para la gestión de ejercicios y rutinas de entrenamiento. La motivación principal es dar una herramienta técnica que automatice la selección de ejercicios bajo criterios de intensidad y tiempo, garantizando la variedad mediante una regla de rotación .

El objetivo es aplicar los pilares de la Programación Orientada a Objetos : abstracción, herencia, encapsulamiento y polimorfismo, logrando un código escalable.

Descripción de la Solución



Modelo Propuesto y Clases Diseñadas

La arquitectura se divide en una jerarquía de clases para los datos y una clase controladora para la lógica:

Clase: Ejercicio (Base Abstracta): Define la estructura común. Al ser abstracta (posee un método virtual puro GetTipo(), impide la creación de ejercicios "genéricos", obligando a la especialización.

Clases: EjercicioCardio y EjercicioFuerza (Derivadas): Heredan de la clase base. Su función es identificar el tipo de ejercicio, permitiendo que el sistema crezca fácilmente si en el futuro se requieren tipos adicionales.

Clase: GestorRutinas (Gestora): Implementa el patrón de diseño de controlador. Administra un “ std::vector<Ejercicio*> “ y contiene la lógica de negocio para las operaciones CRUD y la generación de rutinas.

Justificación de Decisiones

Polimorfismo: Se utiliza para manejar una colección heterogénea de ejercicios. El GestorRutinas no necesita saber el tipo específico del objeto para mostrar su información, lo que reduce el acoplamiento.

Gestión Dinámica: La decisión de usar punteros (Ejercicio*) permite el uso de memoria dinámica , lo cual es indispensable para el polimorfismo. La limpieza de esta memoria está garantizada en el destructor de la clase gestora.

Encapsulamiento: El uso de atributos protected permite que las clases hijas accedan a los datos necesarios sin exponerlos al exterior, manteniendo la integridad del modelo.

Interfaz de Usuario: El archivo main.cpp implementa un menú interactivo que separa la interfaz de la lógica de datos, facilitando posibles cambios futuros (como migrar a una interfaz gráfica sin tocar la lógica de los ejercicios).

Conclusión

El desarrollo de esta solución permitió consolidar el uso de jerarquías de clases y la gestión de memoria en C++. La implementación de la regla de "rotación semanal" demostró que un diseño bien estructurado permite añadir lógica compleja de forma sencilla.

Se cumplieron los objetivos: el sistema permite el mantenimiento completo de la base de datos de ejercicios y genera rutinas personalizadas de forma eficiente. Como aprendizaje principal, el equipo destaca que la inversión de tiempo en el modelado inicial (diagramas y definición de clases base) simplifica drásticamente la codificación de la lógica de negocio final.